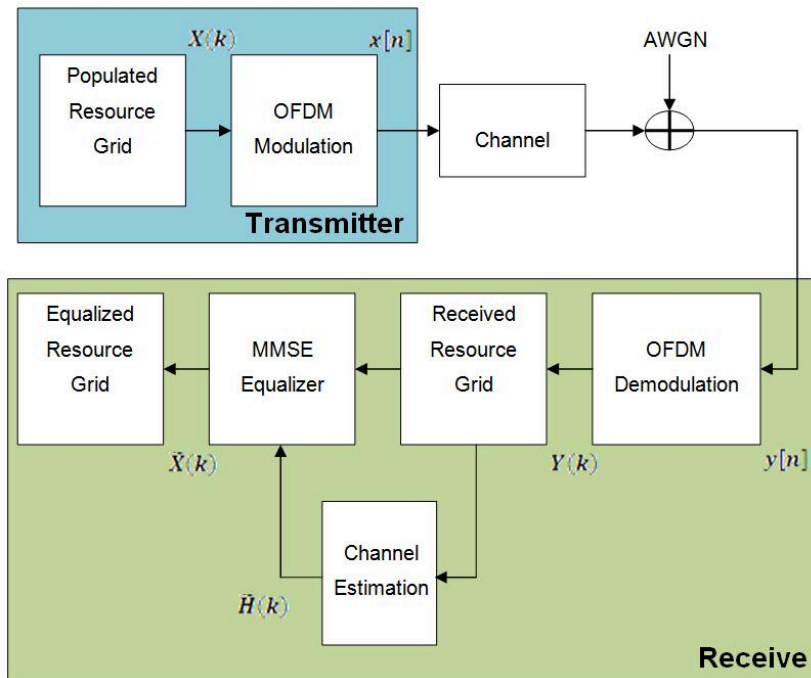


CA Final Project Report

姓名/學號：施政佑/314513032、王瑞程/314513060

1. 環境

本專題的主題為：基於 OFDM receiver 之 LS Channel Estimation + LMMSE/MMSE Equalizer Acceleration



Conceptual OFDM Receiver Background

比較五種不同的平行化模型：

Part	主要平台	Stage 1	Stage 2
Part 1	Scalar RISC-V	Scalar LS channel estimation	Scalar LMMSE equalization
Part 2	RVV	RVV reduction LS channel estimation	RVV LMMSE equalization
Part 3	RVV	SIMD-like RVV LS channel estimation	RVV LMMSE equalization
Part 4	CUDA GPU	CUDA shared-memory LS reduction	CUDA LMMSE equalization
Part 5	CUDA GPU	Multi-pattern CUDA LS reduction	Multi-pattern CUDA LMMSE equalization

實驗環境分成兩類：

gem5 / RISC-V 環境

- Cross compiler : `riscv64-linux-gnu-g++`
- 模擬器 : `gem5`
- CPU model : `TimingSimpleCPU`
- Memory : `256 MB`
- L1 I-cache : `32 kB` , 8-way
- L1 D-cache : `32 kB` , 8-way
- Cache line : `32 bytes`

CUDA 環境

- GPU : `NVIDIA GeForce RTX 3060`
- Compute Capability : `8.6`
- Compiler : `nvcc 12.4`
- CUDA build target : `arch=sm_86`
- Profiling tool : `Nsight Compute (ncu --set basic)`

主要編譯設定如下：

範圍	主要編譯設定
Part 1	<code>riscv64-linux-gnu-g++ -static -O2 -std=c++11</code>
Part 2 / Part 3	<code>riscv64-linux-gnu-g++ -static -O2 -std=c++11 -march=rv64gcv -mabi=lp64d</code>
Part 4 / Part 5	<code>nvcc -O2 -std=c++17 -arch=sm_86 -lineinfo -Xptxas -v</code>

題目在 [reference/CA_Final_Project.pdf](#) 中提醒：

1. GPU 型號需要註明。
2. `gem5 simulated time` 與實際 CPU/GPU runtime 是不同概念，不可直接混用。
3. 運算結果必須輸出或參與後續驗證，避免被 compiler 移除。
4. 報告不能只有數據，必須解釋 performance behavior。

本報告遵循這些原則：

- Part 1~3 使用 `gem5 simulated statistics`。
- Part 4~5 使用 `cudaEvent` 量測的 GPU kernel-only timing。
- 兩類數據分開整理，不直接把 `simSeconds` 與 GPU kernel time 放在同一張快慢比較表中。

2. 檔案說明

```
.
├─ Overall_Results_Analysis.md
├─ README.md
├─ audit_logs
├─ common
├─ data
├─ data_gen
├─ docs
├─ figures
├─ part1
├─ part2
├─ part3
├─ part4
├─ part5
├─ reference
└─ tools
```

本 repository 的主要結構如下：

路徑	內容
<code>common/</code>	共用參數、資料陣列、I/O、模型與驗證函式
<code>data_gen/</code>	單一 pattern OFDM 輸入產生器
<code>data/</code>	產生出的 binary input

路徑	內容
<code>part1/</code>	Scalar baseline
<code>part2/</code>	RVV reduction implementation
<code>part3/</code>	SIMD-like RVV implementation
<code>part4/</code>	Single-pattern CUDA SIMT implementation
<code>part5/</code>	Multi-pattern CUDA implementation
<code>figures/</code>	模擬圖表
<code>reference/</code>	RVV spec 與助教參考資料

實驗相關的主要檔案如下：

檔案	用途
<code>data_gen/generate_ofdm_data.cpp</code>	產生 <code>data/ofdm_input.bin</code>
<code>part5/generate_ofdm_multi.cpp</code>	產生 <code>data/ofdm_input_multi.bin</code>
<code>part1/main.cpp</code>	Part 1 scalar baseline
<code>part2/main.cpp</code>	Part 2 RVV reduction + RVV LMMSE
<code>part3/main.cpp</code>	Part 3 SIMD-like RVV + RVV LMMSE
<code>part4/main.cu</code>	Part 4 CUDA single-pattern implementation
<code>part5/main.cu</code>	Part 5 CUDA multi-pattern implementation
<code>part5/main_cpu.cpp</code>	Part 5 CPU baseline
<code>part1/part2/part3/Makefile</code>	gem5 / RISC-V build and run flow
<code>part4/Makefile</code>	CUDA build, PTX, NCU, sweep
<code>part5/Makefile</code>	Multi-pattern CUDA build, PTX, NCU, CPU baseline, sweep

3. 演算法內容

3.1 OFDM workload

本專題實作的是 pilot-based OFDM receiver pipeline，包含兩段主要計算：

1. `LS channel estimation`
2. `one-tap LMMSE equalization`

固定參數如下：

Parameter	Value	說明
<code>NUM_SUBCARRIERS</code>	<code>512</code>	frequency-domain subcarriers
<code>NUM_PILOTS</code>	<code>256</code>	每個 subcarrier 的 repeated pilot observations
<code>NUM_DATA_SYMBOLS</code>	<code>512</code>	OFDM data symbols
<code>TOTAL_PILOT</code>	<code>131072</code>	<code>512 x 256</code> complex pilot samples
<code>TOTAL_DATA</code>	<code>262144</code>	<code>512 x 512</code> complex equalization outputs
Modulation	<code>QPSK</code>	<code>±1 ± j1</code>
<code>SYMBOL_POWER</code>	<code>2</code>	QPSK symbol power
<code>NOISE_STD</code>	<code>0.0404</code>	AWGN 每個 real dimension 的標準差

資料布局由程式固定為：

```
pilot_index(k, p) = k * NUM_PILOTS + p
data_index(s, k)  = s * NUM_SUBCARRIERS + k
```

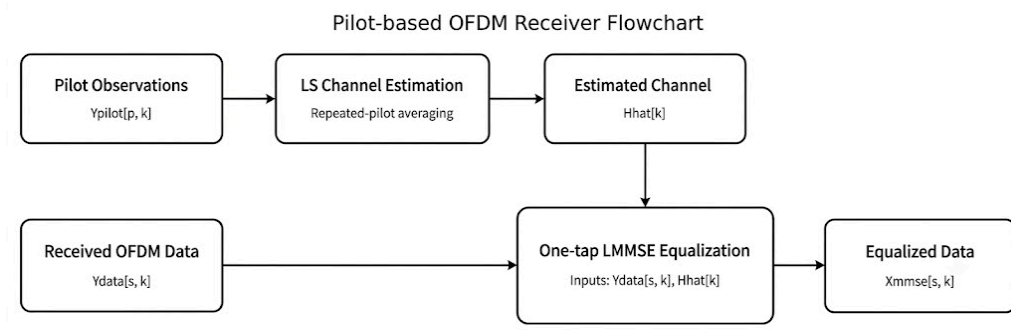
- 固定 k 、沿 p 掃描時，pilot data 是 contiguous。
- 固定 p 、跨不同 k 掃描時，stride 為 `NUM_PILOTS * sizeof(float) = 1024 bytes`。

For Part 2 / Part 3 的 mapping：

- Part 2 適合沿 p 做 unit-stride vector reduction
- Part 3 若讓 lane 對應不同 k ，須承擔 `vlse32.v` 的 strided access

為避免資料布局、程式索引與數學符號不一致，以下全文統一使用 $X_{\text{pilot}}[k, p]$ 、 $Y_{\text{pilot}}[k, p]$ 的表示法。

3.2 數學模型



- Pilot symbols 固定為： $X_{\text{pilot}}[k, p] = 1 + j0$
- Received pilot model： $Y_{\text{pilot}}[k, p] = H[k] + N_{\text{pilot}}[k, p]$
- LS Channel Estimation：

$$\hat{H}[k] = \sum_{p=0}^{P-1} Y_{\text{pilot}}[k, p] \cdot w[p], \quad w[p] = \frac{1}{P}$$

$\therefore$ pilot amplitude = 1
- Real 與 imaginary components：
 - $\hat{H}_r[k] = \sum_p Y_{\text{pilot},r}[k, p] \cdot w[p]$
 - $\hat{H}_i[k] = \sum_p Y_{\text{pilot},i}[k, p] \cdot w[p]$
- 對應 weighted summation $\Rightarrow$ reduction kernel，符合作業對 Part 1 / Part 2 所要求的形式：

$$f(a, b) = a \times b$$

- $a[p] = Y_{\text{pilot}}[k, p]$
- $b[p] = \text{pilot_w}[p]$

- OFDM data model 為： $Y_{\text{data}}[s, k] = H[k]X_{\text{data}}[s, k] + N_{\text{data}}[s, k]$
- LMMSE/MMSE-form equalizer： $\hat{X}_{\text{mmse}}[s, k] = \frac{Y_{\text{data}}[s, k] \cdot \hat{H}^*[k]}{|\hat{H}[k]|^2 + \sigma_n^2 / \sigma_x^2 + \epsilon}$
- 其 real-valued implementation：
 - $D[k] = \hat{H}_r[k]^2 + \hat{H}_i[k]^2 + \text{NOISE_VAR_OVER_SYMBOL_POWER} + \epsilon$
 - `EPSILON`：避免 denominator 在 deep fade 時過小而造成 numerical instability
 - $\hat{X}_r[s, k] = \frac{Y_r[s, k] \hat{H}_r[k] + Y_i[s, k] \hat{H}_i[k]}{D[k]}$
 - $\hat{X}_i[s, k] = \frac{Y_i[s, k] \hat{H}_r[k] - Y_r[s, k] \hat{H}_i[k]}{D[k]}$

element-wise data-parallel kernel $\Rightarrow$ 以 RVV 的 unit-stride vector operations 進行加速。

3.3 Part 5：multi-pattern

Part 5 把同一套運算複製到多個 patterns。即不同 patterns 之間具有：

- 不同的 QPSK data symbols
- 不同的 AWGN realizations
- 相同的 static channel realization

⇒ Part 5 處理的是 shared static channel 下、多個 symbols 與 noise realizations 不同的 OFDM packets。

引入 pattern index m ：

```
Hhat[m,k] = sum_p Y_pilot[m,k,p] * w[p]

Xmmse[m,s,k] = Ydata[m,s,k] * conj(Hhat[m,k])
                / (|Hhat[m,k]|^2 + NOISE_VAR_OVER_SYMBOL_POWER + EPSILON)
```

4. 實現方法

4.1 輸入資料與可重現性

Part 1~4 使用相同的單一 pattern 輸入：

- generator：data_gen/generate_ofdm_data.cpp
- binary：data/ofdm_input.bin

Part 5 使用 multi-pattern 輸入：

- generator：part5/generate_ofdm_multi.cpp
- binary：data/ofdm_input_multi.bin

⇒資料可重現：generator 都使用固定 seed 與固定參數

4.2 Part 1：Scalar baseline

Part 1：formal scalar baseline，不使用 RVV 或 CUDA。它保留兩段主要 computation stages：

1. estimate_channel_ls_average_scalar()
2. equalize_lmmse_scalar()

Stage 1 的外層迴圈掃過 subcarrier k ，內層掃過 pilot p ，形成 nested-loop weighted summation。符合：

- nested loops
- reduction-like arithmetic form

4.3 Part 2：RVV reduction + RVV LMMSE

Part 2 的 Stage 1 維持與 Part 1 相同的數學，但把同一個 k 的不同 pilot observations p 映射到 RVV lanes，再用 vector reduction 指令把結果加總為一個 $Hhat[k]$ 。

對應 mapping：

```
vector lanes -> different p for the same k
reduction    -> sum over p
```

Stage 2 則沿著 subcarrier k 做 unit-stride RVV element-wise vectorization

4.4 Part 3：SIMD-like RVV + RVV LMMSE

Part 3 的 Stage 1 改成 across- k SIMD-like mapping

- 不使用 vector reduction
- 每個 lane 對應不同的 output subcarrier k

- 固定 `p`，同時更新多個 `Hhat[k]`
- 使用 `vlse32.v` 進行 strided load

對應 mapping：Part 3 的 lanes 代表多個彼此獨立的輸出

```
vector lanes    -> different output k
no reduction
strided access -> vlse32.v
```

4.5 Part 4 : CUDA SIMT single-pattern

Part 4 將相同 pipeline 移到 GPU，處理單一 OFDM input pattern。

Stage 1 kernel：

```
ls_channel_estimation_shared_kernel()
```

mapping：

- one block → one subcarrier `k`
- each thread 累積一個或多個 pilot partial contributions
- pilot mapping：`p = tid, tid + blockDim.x, tid + 2 * blockDim.x, ...`
- block 內使用 `__shared__` 做 block-level tree reduction

Stage 2 kernel：

```
lmmse_equalization_kernel()
```

mapping：

- one thread → one output `Xmmse[s,k]`
- `idx = blockIdx.x * blockDim.x + threadIdx.x`

對應：

- CUDA SIMT execution
- `threadIdx.x / blockDim.x / blockDim.x`
- `__shared__`
- PTX / PTXAS / NCU analysis

4.6 Part 5 : multi-pattern CUDA

Part 5 在 Part 4 的基礎上新增 pattern dimension，使用 2D CUDA grid：

Stage 1：

- `blockIdx.x -> subcarrier k`
- `blockIdx.y -> pattern index`
- `threadIdx.x -> pilot contributions`

Stage 2：

- `blockIdx.x * blockDim.x + threadIdx.x -> flattened output index inside one pattern`
- `blockIdx.y -> pattern index`

→ 使 GPU 同時看到更多獨立 blocks 與 warps。

5. 驗證方法與量測

5.1 Correctness check

所有 parts 都會輸出以下驗證量：

Metric	用途
H_MSE	驗證 channel estimation
MSE_RX_BEFORE_EQ	未等化前 baseline
MSE_LMMSE	驗證 equalization 成效
checksum	防止 compiler 移除運算

Verification 條件為：

```
MSE_LMMSE < MSE_RX_BEFORE_EQ
H_MSE < 0.01
checksum != 0
```

避免未使用的運算被最佳化移除

5.2 Performance measurement

Part 1~3 使用 gem5，量測的是 receiver executable 的 simulated program statistics。統計包含：

- binary input loading
- LS / LMMSE computation stages
- correctness checks
- checksum
- 結果輸出

不包含獨立執行的 host-side input generator。

Part 4~5 使用 `cudaEvent`，量測的是：GPU kernel-only timing

因此 gem5 的 `simSeconds` 不直接和 GPU `PIPELINE_KERNEL_MS` 比較

6. 模擬結果說明

6.1 Part 1~Part 3 gem5 結果

三個版本的 correctness 都通過：

Part 1~Part 3 皆使用同一份 single-pattern OFDM input，並在 gem5 下量測完整程式執行。三個版本的 correctness 皆通過，且 `H_MSE`、`MSE_RX_BEFORE_EQ` 與 `MSE_LMMSE` 幾乎一致，表示 scalar、RVV reduction 與 SIMD-like RVV 三種實作在數學演算法上對齊。

Part	H_MSE	MSE_RX_BEFORE_EQ	MSE_LMMSE	Verification
Part 1	0.00001250	0.14093372	0.00681053	PASS
Part 2	0.00001250	0.14093372	0.00681052	PASS
Part 3	0.00001250	0.14093372	0.00681053	PASS

gem5 比較如下：

性能方面，Part 2 是 Part 1~3 中最快的版本。相較 Part 1 將 `simSeconds` 與 `numCycles` 都降到約原先的八成，`simInsts` 則下降約三分之一，即 Stage 1 的 reduction mapping 在目前設計下能帶來明顯的 instruction-count reduction，並進一步反映到模擬時間與 cycle 數上。

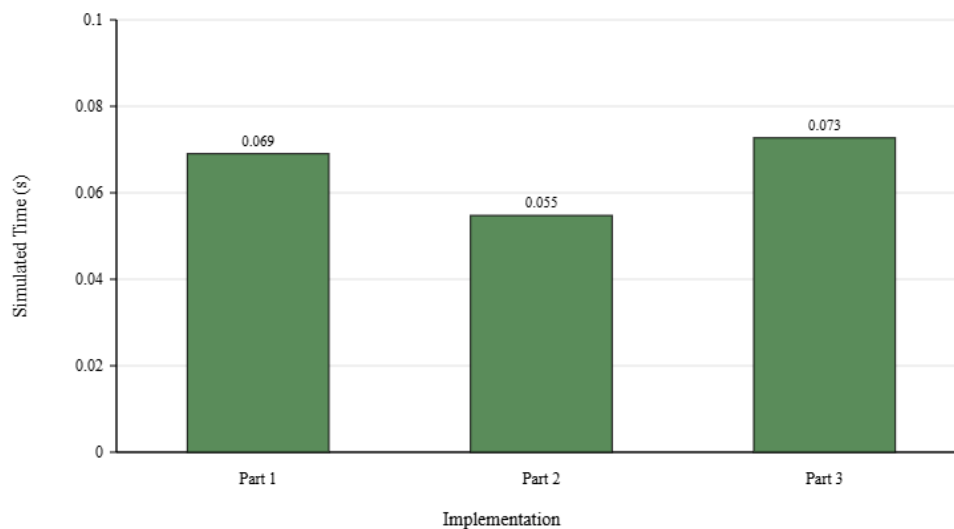
相較之下，Part 3 雖然同樣減少了 `simInsts`，但 `simSeconds` 與 `numCycles` 反而高於 Part 1，這顯示 instruction 數不是唯一決定效能的因素。

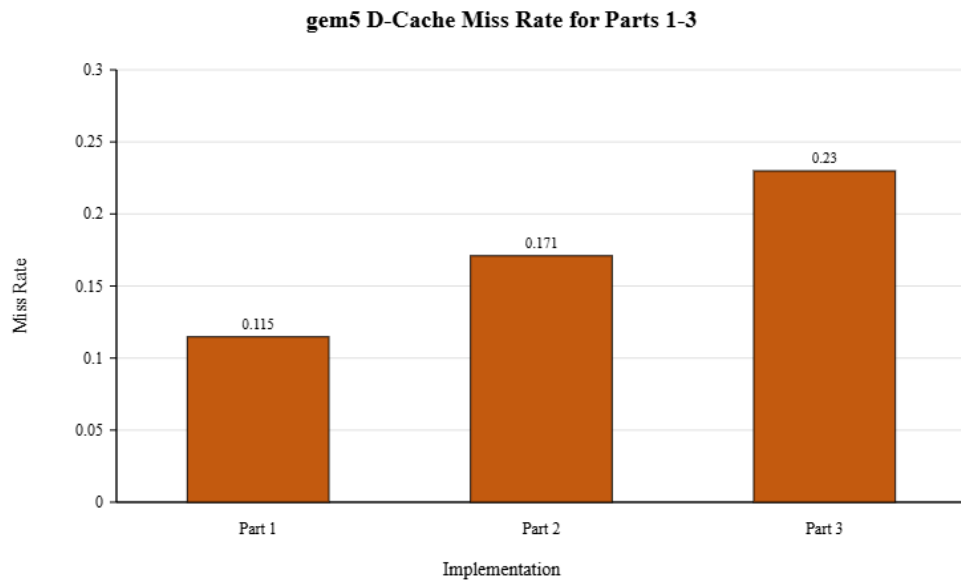
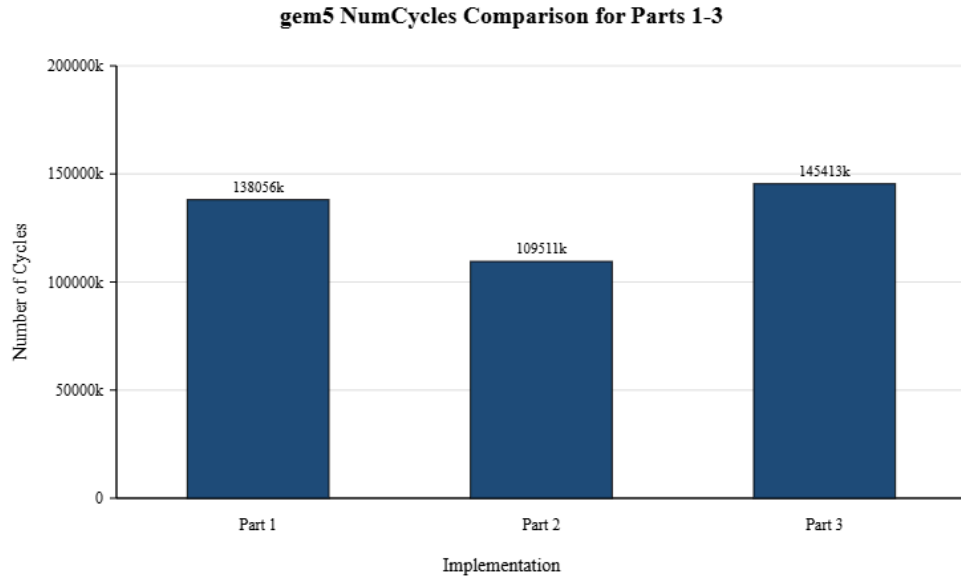
- Part 2 是 Part 1~3 中最快的版本。
- Part 2 相較 Part 1：
 - `simSeconds` 下降約 20.68%
 - `numCycles` 下降約 20.68%
 - `simInsts` 下降約 33.23%
- Part 3 雖然 `simInsts` 也下降，但 `simSeconds` 反而比 Part 1 高約 5.33%。
 - Part 2 用的是 contiguous pilot dimension 上的 vector reduction，因此能有效減少指令
 - Part 3 則必須遵守：
 - no reduction
 - across-`k` SIMD-like mapping
 - strided memory access

在目前 `pilot_index(k, p) = k * NUM_PILOTS + p` 下，固定 `p`、跨 `k` 的 stride 為 1024 bytes。這讓 Part 3 的 `vlse32.v` 載入模式較不利於 spatial locality，因此 `D-cache miss rate` 升到 0.229838，`CPI` 也提高到 12.973001。

Metric	Part 1 Scalar	Part 2 RVV	Part 3 SIMD-like RVV
<code>simSeconds</code>	0.069028	0.054755	0.072706
<code>simInsts</code>	17,066,251	11,395,368	11,208,851
<code>numCycles</code>	138,055,892	109,510,852	145,412,866
<code>CPI</code>	8.089394	9.610092	12.973001
<code>IPC</code>	0.123619	0.104057	0.077083
<code>D-cache miss rate</code>	0.114745	0.170969	0.229838
<code>I-cache miss rate</code>	0.000046	0.000071	0.000051

gem5 SimSeconds Comparison for Parts 1-3





6.2 Part 4 結果

Part 4 將同樣的 OFDM 移到 CUDA GPU，先測試 single-pattern 預設下的行為。這邊量測使用 `cudaEvent`，即數字代表 GPU kernel-only timing，不包含 input loading、memory allocation、H2D / D2H copy 與 host-side verification。預設 case 使用 shared-memory LS reduction 與 `TPB_LS=256`、`TPB_EQ=256`，在此設定下 correctness 維持 `PASS`，且 Stage 1 與 Stage 2 的時間已經落在相近量級。

```
TPB_LS = 256
TPB_EQ = 256
LS_KERNEL_MODE = shared
```

對應 correctness 與 timing：

Metric	Value
H_MSE	0.00001250
MSE_RX_BEFORE_EQ	0.14093372
MSE_LMMSE	0.00681053

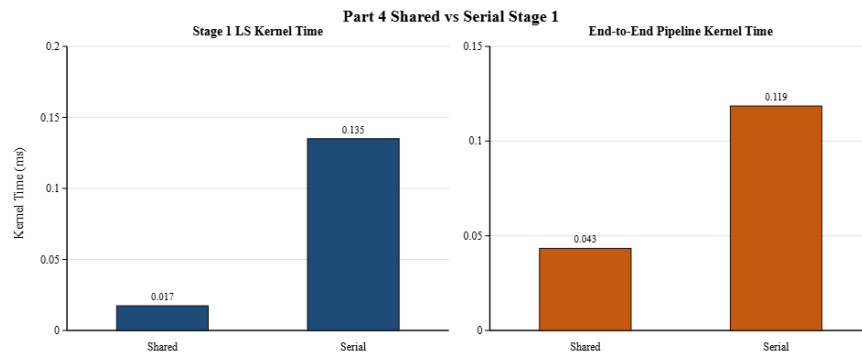
Metric	Value
checksum	584.37121582
Verification	PASS
LS_KERNEL_MS	0.023186
LMMSE_KERNEL_MS	0.024146
PIPELINE_KERNEL_MS	0.044678

Part 4 shared-vs-serial Stage 1 比較如下：

shared-memory Stage 1 與 one-thread-per-subcarrier 的 serial baseline，可看出 block-cooperative reduction 中 shared 版本不只是把資料放進 `__shared__`，而是讓同一個 block 內的 threads 共同完成對 256 個 pilots 的 reduction，因此能把 Stage 1 的工作切成多個 partial sums，再以 tree reduction 合併。

- 使 `ls_shared_256` 的 pipeline time 明顯低於 `ls_serial_256`
- shared memory 存在明確的 block-level cooperation 需求

Case	LS Mode	LS ms	Pipeline ms	Verification
<code>ls_shared_256</code>	shared	0.017372	0.043345	PASS
<code>ls_serial_256</code>	serial	0.135022	0.118535	PASS



Part 4 TPB sweep 摘要：

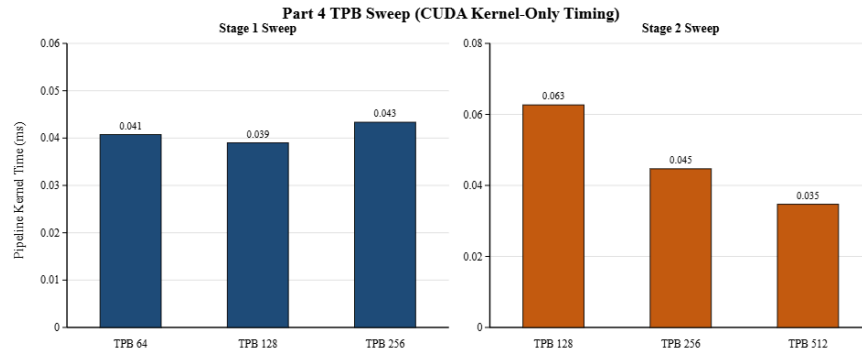
在 block size sweep，結果顯示 `TPB_LS` 與 `TPB_EQ` 都不是越大越快

- 以 isolated LS kernel time 而言，`TPB_LS=128` 最快
- 以 isolated LMMSE kernel time 而言，`TPB_EQ=128` 略快
- 以整體 pipeline time 而言，`TPB_EQ=512` 最佳

代表 block size 會同時影響 scheduler 可見的 threads / warps 數量、register 與 shared memory 資源分配、以及不同 kernel 的 memory behavior，因此最終性能呈現 Non-monotonic 結果，而非單純由 thread 數決定。

這也代表 Stage 1 與 Stage 2 的最佳 TPB 不一定相同，整體 pipeline 最佳配置也不一定等於單一 kernel 的最佳配置。

Case	TPB_LS	TPB_EQ	LS Mode	LS ms	LMMSE ms	Pipeline ms
<code>ls_shared_64</code>	64	256	shared	0.017032	0.019973	0.040740
<code>ls_shared_128</code>	128	256	shared	0.015666	0.020561	0.039000
<code>ls_shared_256</code>	256	256	shared	0.017372	0.020174	0.043345
<code>eq_shared_128</code>	256	128	shared	0.024464	0.021142	0.062669
<code>eq_shared_256</code>	256	256	shared	0.023186	0.024146	0.044678
<code>eq_shared_512</code>	256	512	shared	0.035142	0.021271	0.034696



PTXAS 與 NCU 的結果也支持此現象

- Stage 1 shared kernel 使用 `14 registers/thread`、`0 spill`，dynamic shared memory 約 `2.05 KB/block`
- Stage 2 LMMSE kernel 則使用 `21 registers/thread`、同樣沒有 spill

從 NCU 來看

- Stage 1 的 throughput 約落在 `36%` 左右，並伴隨 shared-memory traffic 與 synchronization
- Stage 2 的 memory throughput 約 `79%`、compute throughput 約 `30%`，更接近 memory-heavy kernel 的樣貌

6.3 Part 5 結果

Part 5 是在 Part 4 的基礎上加入 pattern dimension，讓 GPU 同時處理多個 OFDM patterns。預設 case 採用 `NUM_PATTERNS=16`，Stage 1 與 Stage 2 都維持 CUDA GPU 執行，但 block mapping 多了一個 pattern 軸，使同時 active 的 blocks 與 warps 增加。

從 correctness 來看，GPU 與 CPU baseline 對同一份 multi-pattern input 都得到 `PASS`，`H_MSE` 與 `MSE_LMMSE` 也一致，表示多 pattern 延伸沒有改變演算法本身。

Part 5 的預設 multi-pattern case 為：

```
NUM_PATTERNS = 16
TPB_LS = 256
TPB_EQ = 256
```

GPU correctness 與 kernel-only timing：

Metric	Value
<code>H_MSE</code>	<code>0.00001289</code>
<code>MSE_RX_BEFORE_EQ</code>	<code>0.14082038</code>
<code>MSE_LMMSE</code>	<code>0.00682142</code>
<code>checksum</code>	<code>9294.77246094</code>
<code>Verification</code>	<code>PASS</code>
<code>LS_KERNEL_MS</code>	<code>0.128993</code>
<code>LMMSE_KERNEL_MS</code>	<code>0.308572</code>
<code>PIPELINE_KERNEL_MS</code>	<code>0.434780</code>

同一份 multi-pattern input 的 CPU baseline：

Metric	Value
<code>CPU_PIPELINE_MS</code>	<code>12.268151</code>
<code>H_MSE</code>	<code>0.00001289</code>
<code>MSE_LMMSE</code>	<code>0.00682142</code>

Metric	Value
Verification	PASS

把同一份 multi-pattern input 的 CPU baseline 一起看，可看出 GPU throughput 優勢。以 16 patterns 為例，GPU kernel-only pipeline time 為 0.434780 ms，CPU baseline 則為 12.268151 ms，約有 28.22x 的差距。

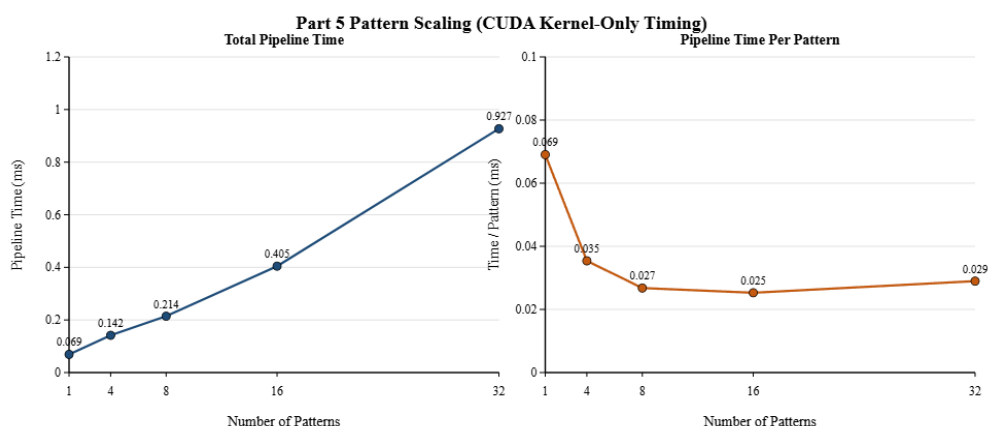
注意：本節前面的預設 case PIPELINE_KERNEL_MS = 0.434780 與下表 pattern sweep 中 Patterns = 16 的 Pipeline ms = 0.404536 來自不同量測紀錄；前者作為預設 case 單點結果，後者作為 sweep 中的對應資料點，因此不可視為同一次量測或直接逐項對應。若要嚴格並列比較，應在表中一併標示 warmup、timed iterations 等設定。

Pattern sweep：

Pattern sweep 顯示隨著 patterns 從 1 增加到 16，總 pipeline time 上升，因為總工作量變多。但 ms / pattern 會從 0.069048 降到 0.025284，表示固定 launch、scheduling 與部分 pipeline overhead 被更多工作量 share。當 patterns 繼續增加到 32 時，ms / pattern 又回升到 0.028968，

→ throughput 改善並非無限線性，當 active patterns 過多時，memory traffic 與整體資源利用開始成為新的 restriction。

Patterns	Verification	LS ms	LMMSE ms	Pipeline ms	ms / pattern
1	PASS	0.032481	0.037750	0.069048	0.069048
4	PASS	0.039578	0.117176	0.141583	0.035396
8	PASS	0.059996	0.177853	0.214241	0.026780
16	PASS	0.086354	0.347571	0.404536	0.025284
32	PASS	0.223770	0.745861	0.926991	0.028968



7. 討論與比較

7.1 Part 3 Think：

Part 1~Part 3 的主要差異在 Stage 1 的 execution mapping。

- Part 1 是 scalar nested-loop baseline
- Part 2 讓 vector lanes 對應同一個 k 的不同 p ，再以 reduction 合併成單一 $\hat{H}[k]$
- Part 3 則讓 vector lanes 對應不同的 k ，在固定 p 的情況下同步更新多個輸出

問題 1：為什麼 cycle reduction 與 instruction reduction 不完全相同？

回答：從實驗結果來看，Part 3 的 simInsts 比 Part 1 少，但 numCycles 與 simSeconds 反而更高。這表示 cycles 不只由 instruction count 決定，還會受到 memory latency、cache miss、pipeline stalls 與 CPI 影響。在實驗設計中，Part

3 的 across-k mapping 需要使用 `vlse32.v` 做 strided load，使 `D-cache miss rate` 從 Part 1 的 `0.114745` 升到 `0.229838`，`CPI` 也提高到 `12.973001`。因此 instruction 減少並沒有自然轉換成 cycle 的下降。

問題 2：當 k 愈大，performance 愈好嗎？為什麼？

從實驗結果來看不一定。對 Part 3 來說，增加同時處理的 lane 數可以減少 scalar loop overhead，但同時受限於 RVV 可用的實際 vector length，以及 strided memory access 帶來的 locality penalty。在目前 gem5 設定下，`VLEN=256` 且 `SEW=32`，最多同時處理 8 個 FP32 lanes，再往上不是單次更大的 vector，而是更多 strip-mined iterations。

因此即使理論上 lanes 變多，若記憶體行為更差、partial sum 需要更多讀寫，效能也不一定跟著改善。因此更大的 `k` 或更多 lanes 並不保證更好的 performance。

問題 3：為什麼 Part 3 不需要 reduction operations？與 Part 2 相比有什麼差異或取捨？

參考 lane 的定義。Part 2 的每個向量代表同一個輸出的多個被加總元素，因此最後一定要把 lanes 收斂成單一 scalar，所以需要 `vfredusum.vs` 這類 vector reduction。Part 3 的每個 lane 則代表不同的輸出 subcarrier `k`，每個 lane 自己維護 partial sum，lane 之間沒有彼此合併的需求，因此不需要 reduction operations。相較 Part 2，Part 3 的優點是符合的 SIMD-like across-iteration mapping，但代價是須承擔 `vlse32.v` strided access 帶來的 locality penalty。在目前資料設計下，Part 2 因為使用 unit-stride load 與真正的 vector reduction，所以式 Part 1~3 中最快。

7.2 Part 4 Think：

問題 1：Shared memory 在什麼情況下比較有效？使用與否會造成什麼影響？

從實驗結果來看 shared memory 對 Stage 1 最有效，因為需要 block 內 threads 共同完成 reduction。即每個 thread 先計算自己的 pilot partial sum，再把結果放進 shared memory 做 tree reduction，這比讓每個 thread 獨立完成全部 serial reduction 更符合 GPU 的執行模式。

實驗：`ls_shared_256` 的 pipeline time 為 `0.043345 ms`，而 `ls_serial_256` 則為 `0.118535 ms`，可看到 shared-memory reduction 有效。為 block-cooperative reduction、shared memory 與 thread parallelism 的整體效果。對 Stage 2 的 element-wise LMMSE kernel 而言，由於每個 thread 的資料重用較低，使用 shared memory 的必要性就較小。

問題 2：Block size (Threads Per Block) 愈大，performance 一定愈好嗎？

從目前 TPB sweep 可以看到不一定

- 若看 isolated LS kernel time，`TPB_LS=128` 最快
- 若看 isolated LMMSE kernel time，`TPB_EQ=128` 略快
- 若看整體 pipeline time，則是 `TPB_EQ=512` 最佳，結果明顯不是單調的。

表示 block size 需在 thread-level parallelism、resource usage、scheduler behavior 與 memory access 之間 tradeoff，而不是單純愈大愈快。過小的 block 可能讓可用平行度不足，過大的 block 則可能增加 coordination 成本。

問題 3：Block size 會影響 GPU 同時執行的 threads/warps 數量嗎？occupancy 如何改變？

Yes。每個 block 的 thread 數、register 使用量與 shared memory 用量，都會決定一個 SM 上能同時 resident 的 blocks / warps 數量。

如 Part 4 的 Stage 1 shared kernel 需要約 `2.05 KB` 的 dynamic shared memory，而 Stage 2 的 registers per thread 則更高

⇒ resource 都會影響 occupancy。當 block size 改變時，同一個 SM 可同時容納的 blocks 數量也可能改變，因此 occupancy 不會固定不變，而是和 block 配置與 kernel 資源需求一起變動。

問題 4：Occupancy 的改變會如何影響 GPU latency hiding 能力？

occupancy 提升通常有助於 latency hiding，因為當部分 warps 因記憶體延遲而停住時，scheduler 還能切換到其他可執行的 warps，以減少閒置週期。不過若過高 occupancy 而導致 memory traffic 更差、register 壓力增加，或 block coordination 開銷更高，未必會得到更快的 kernel。

Part 4 的結果說明 occupancy 與 performance 有關，但兩者並非 Monotonic 對應關係。

7.3 Part 5 Think : multi-pattern GPU parallelism

Part 5 把 single-pattern pipeline 延伸到 multi-pattern，因此題目希望觀察 GPU scalability 與 execution behavior。以下依照 PDF 中的 Think 問題逐一回答。

問題 1：為什麼 Part 5 會比 Part 4 更適合 GPU？

GPU 擅長處理大量彼此獨立的 threads 與 warps。單一 pattern 的 workload 雖然已具平行性，但獨立 blocks 數量仍有限，加入 pattern dimension 之後，`blockIdx.y` 對應不同 patterns，GPU 同時看到的 block 數量增加，因此更能發揮 throughput-oriented architecture 的特性。

⇒ Part 5 的新增平行性不是來自單一 OFDM frame 內部，而是來自不同 patterns 之間彼此獨立，可同時排入 GPU 執行。

問題 2：為什麼 GPU 必須依賴大量 threads/warps 才能發揮 scalability 的效益？

GPU 的 latency hiding 依賴 warp scheduling。當某些 warps 因記憶體存取而等待時，SM 必須有其他可執行的 warps 來填補空槽。若活躍 threads / warps 太少，記憶體延遲與 pipeline bubble 就更難被隱藏，GPU 利用率自然下降。

⇒ Part 5 的利用多個 patterns 讓同時活躍的 blocks 與 warps 增加，因此比 Part 4 更容易展現 GPU 的 scalability 與 throughput 優勢。

問題 3：增加 pattern 數量後，performance 是否持續提升？會隨數量線性成長嗎？

不會線性成長。從目前 sweep 可見，總 pipeline time 會隨 patterns 增加而上升，因為總工作量變大；但 `ms / pattern` 會先從 1 pattern 的 0.069048 降到 16 patterns 的 0.025284。不過到 32 patterns 時，`ms / pattern` 又回升到 0.028968，說明 throughput 改善並不是無限線性，而是逐漸受限於 memory traffic 與 resource contention。

問題 4：增加 pattern 數量如何影響 occupancy 與 latency hiding 能力？進而對 GPU utilization 造成什麼影響？

當 pattern 數增加時，grid 內會有更多 blocks，同時給 scheduler 更多可選的 warps，因此在 pattern 數不大時有助於 occupancy 與 latency hiding，也解釋為什麼 per-pattern time 會下降。然而當 patterns 繼續增加。

Stage 2 這類 memory-heavy kernel 會逐漸受限於 DRAM throughput 與 memory subsystem。NCU 結果也顯示：Stage 2 的 memory throughput 已達約 91.45%，表示 GPU utilization 的改善最後會逐漸接近平台限制，而不是無限制成長。

8. 結論

期末專題設計以同一個 pilot-based OFDM receiver 系統比較 scalar、RVV 與 CUDA 三種不同層次的 parallel execution model。從 Part 1~Part 3 的 gem5 結果來看，Part 1 提供了穩定的 scalar reference，Part 2 透過 `vfredusum.vs` reduction 在目前設定下得到最佳效能，而 Part 3 雖然使用 SIMD-like RVV mapping，卻因為題目要求 no reduction 與 strided access，使 memory behavior 不佳，最後呈現 instruction 降低但 cycles 不降反升的現象。

從 Part 4 與 Part 5 的 CUDA 結果來看，block-cooperative shared-memory reduction 能有效支撐單一 pattern 的 Stage 1，而 Stage 2 則更明顯呈現 memory-heavy kernel 的特性。當同樣的 pipeline 延伸到 multi-pattern workload 時，GPU 同時能看到更多獨立 blocks 與 warps，展現出明顯的 overhead amortization 與 throughput 優勢。但也知道改善並非無限制線性成長，而是會逐漸受到 memory traffic、occupancy 與整體資源利用的限制。